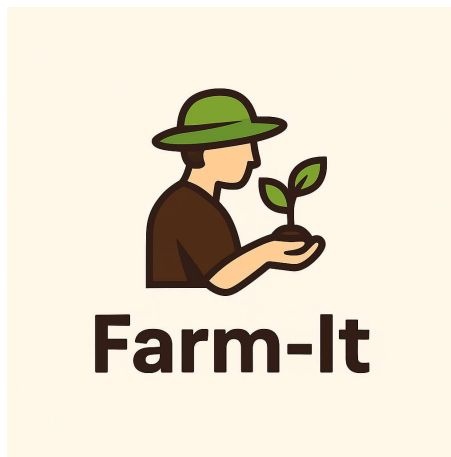


Computer Science

FARM-IT



CONCEPTUAL DESIGNERS

Group Members:

1. Jaswant Sreram P
2. Akshan R
3. SharathVijay G S

Table of Contents

Introduction	3
Softwares Used	4
Flow diagram	5
Source Code	6
Application Flow	32
Home Screens Main window Login window Admin dashboard Customer dashboard Farmer dashboard Register window	33
Report Module Admin screen Customer screen Farmer screen Register screen	37
Files part of this project Python file Exe file	44
Bibliography	45

Introduction

Agriculture forms the backbone of the Indian economy, but farmers often face challenges in marketing their produce and receiving fair prices due to the presence of intermediaries. To address this issue, **Farm-It** has been developed as a Python-based desktop application that enables farmers to directly showcase their products to customers, eliminating middlemen and maximizing their profits.

Farm-It provides a simple, user-friendly interface designed using Python's Tkinter module, making it accessible for both farmers and customers. The application allows farmers to upload their product details such as name, type, price, contact information, and location. Customers can search for products using filters such as product name, tags (e.g., #cereals, #vegetables), price range, and location, enabling them to easily find what they need. Additionally, customers can maintain a personalized wishlist for future reference.

The application also provides an **Admin dashboard** where administrators can monitor all farmer and customer accounts, manage product listings, and delete or update any inappropriate entries. All the data—farmer details, customer details, and product information—is stored in a MySQL database, ensuring security and easy access to information.

Farm-It has been built as a **Grade 12 Computer Science project** to demonstrate the integration of Python programming with database connectivity using MySQL. The project not only simplifies agricultural trading but also showcases how technology can empower farmers and improve the agricultural supply chain.

Softwares Used

1. Python 3.13

- Used as the main programming language for developing the Farm-It application.
- Python's simplicity, readability, and vast library support made it ideal for building a GUI-based desktop application.

2. Tkinter

- A built-in Python library used for creating the graphical user interface (GUI).
- Tkinter widgets such as labels, buttons, frames, and entry fields were used to design the farmer, customer, and admin dashboards.

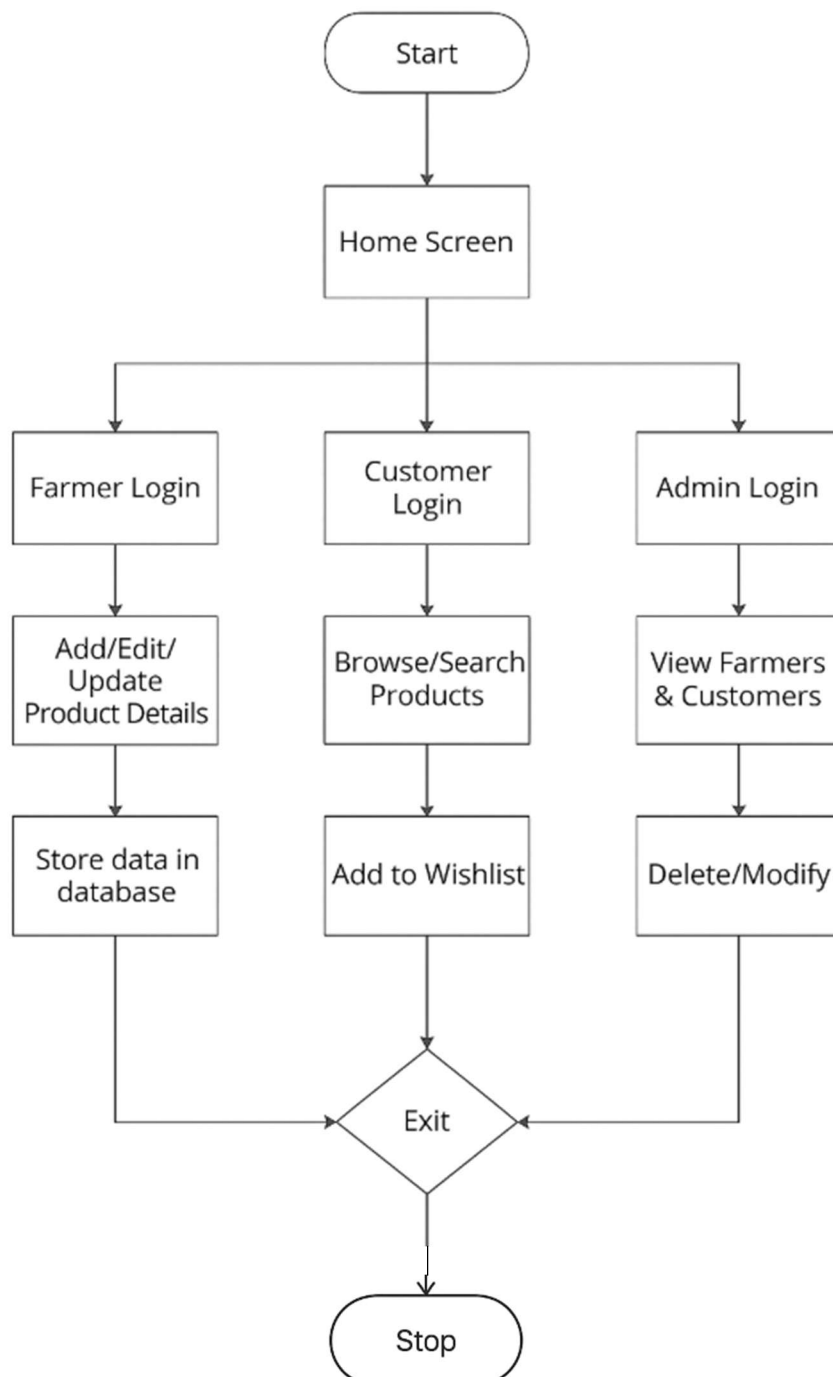
3. MySQL Database

- Used for storing and managing all project data such as farmer profiles, customer details, product listings, and wishlist entries.
- MySQL Connector/Python library was used to establish a connection between the Python application and the MySQL database.

4. PyInstaller

- A tool used to convert the Python scripts into a standalone .exe file, allowing the application to run on any Windows system without requiring Python installation.

Flow Diagram



Source Code:

1. db.py

```
import mysql.connector

DB_CONFIG = {
    'host': 'localhost',
    'user': 'root',
    'password': 'admin',
    'database': 'farmit'
}

def get_connection():
    return mysql.connector.connect(**DB_CONFIG)

def fetch_all(query, values=()):
    conn = get_connection()
    cursor = conn.cursor(dictionary=True)
    cursor.execute(query, values)
    result = cursor.fetchall()
    conn.close()
    return result

def execute_query(query, values=()):
```

```
conn = get_connection()
cursor = conn.cursor()
cursor.execute(query, values)
conn.commit()
conn.close()
```

2. admin.py

```
import tkinter as tk
from tkinter import ttk, messagebox
from db import fetch_all, execute_query
import os
from PIL import Image, ImageTk
```

```
def open_admin_dashboard():
    login = tk.Toplevel()
    image = Image.open(r"loginbg.jpg")
    image = image.resize((500,500), Image.Resampling.LANCZOS)
    bg_image = ImageTk.PhotoImage(image)
    bg_label = tk.Label(login, image=bg_image)
    bg_label.image = bg_image
    bg_label.place(x=0, y=0, relwidth=1, relheight=1)
    login.title("Admin Login")
    login.geometry("500x500")

    tk.Label(login, text="Username", font=("Helvetica", 20)).pack(pady=50)
```

```
username_entry = tk.Entry(login)
```

```
username_entry.pack()
```

```
tk.Label(login, text="Password",font=("Helvetica", 20)).pack(pady=50)
```

```
password_entry = tk.Entry(login, show="*")
```

```
password_entry.pack()
```

```
def authenticate():
```

```
    user = username_entry.get()
```

```
    pw = password_entry.get()
```

```
    result = fetch_all("SELECT * FROM admins WHERE username=%s AND  
password=%s", (user, pw))
```

```
    if result:
```

```
        login.destroy()
```

```
        show_admin_dashboard()
```

```
    else:
```

```
        messagebox.showerror("Login Failed", "Invalid credentials")
```

```
tk.Button(login, text="Login", command=authenticate).pack(pady=30)
```

```
def show_admin_dashboard():
```

```
    window = tk.Toplevel()
```

```
    window.title("Admin Dashboard")
```

```
    window.attributes('-fullscreen', True)
```



```
tk.Button(window, text="Home", command=lambda: [window.destroy(),
os.system("python main.py")], bg="orange", fg="white", font=("Arial",
12)).pack(pady=10)
```

```
style = ttk.Style()
```

```
style.theme_use("default")
```

```
# Change Treeview background
```

```
style.configure("Treeview",
```

```
background="#e8f5e9",    # light green
```

```
foreground="black",
```

```
rowheight=25,
```

```
fieldbackground="#e8f5e9")
```

```
# Change selected row highlight color
```

```
style.map("Treeview",
```

```
background=[("selected", "#a5d6a7")])
```

```
notebook = ttk.Notebook(window)
```

```
notebook.pack(fill=tk.BOTH, expand=True)
```

```
farmer_tab = ttk.Frame(notebook)
```

```
customer_tab = ttk.Frame(notebook)
```

```
notebook.add(farmer_tab, text="Farmers")
```

```
notebook.add(customer_tab, text="Customers")
```

```
def load_table(tab, query, delete_query, update_query=None):
```

```

for widget in tab.winfo_children():
    widget.destroy()

data = fetch_all(query)
cols = list(data[0].keys()) if data else []

tree = ttk.Treeview(tab, columns=cols, show="headings")
for col in cols:
    tree.heading(col, text=col)
    tree.column(col, width=120)

for row in data:
    tree.insert("", "end", iid=row['id'], values=tuple(row.values()))
tree.pack(fill=tk.BOTH, expand=True)

def delete_selected():
    selected = tree.selection()
    for sel in selected:
        execute_query(delete_query, (int(sel),))
    load_all()

def edit_selected():
    selected = tree.selection()
    if not selected:
        return
    row = tree.item(selected[0])['values']

```

```

edit_window = tk.Toplevel()
edit_window.title("Edit Entry")
entries = []
for i, col in enumerate(cols):
    tk.Label(edit_window, text=col).pack()
    e = tk.Entry(edit_window)
    e.insert(0, row[i])
    e.pack()
    entries.append(e)

def save_changes():
    values = [e.get() for e in entries[1:]] # skip id
    values.append(entries[0].get()) # id last
    execute_query(update_query, tuple(values))
    edit_window.destroy()
    load_all()

tk.Button(edit_window, text="Save", command=save_changes).pack()

if update_query:
    tk.Button(tab, text="Edit Selected", command=edit_selected, bg="blue",
fg="white").pack(pady=5)
    tk.Button(tab, text="Delete Selected", command=delete_selected, bg="red",
fg="white").pack(pady=5)

def load_all():

```

```
load_table(farmer_tab, "SELECT * FROM farmers", "DELETE FROM farmers  
WHERE id = %s", "UPDATE farmers SET username=%s, password=%s, name=%s,  
product=%s, contact=%s, tag=%s, location=%s WHERE id=%s")
```

```
load_table(customer_tab, "SELECT * FROM customers", "DELETE FROM  
customers WHERE id = %s")
```

```
load_all()
```

3. customer.py

```
import tkinter as tk  
from tkinter import messagebox, ttk  
from db import fetch_all, execute_query  
import os  
from PIL import Image, ImageTk
```

```
def open_customer_interface():  
    login_window = tk.Toplevel()  
    image = Image.open(r"loginbg.jpg")  
    image = image.resize((500,500), Image.Resampling.LANCZOS)  
    bg_image = ImageTk.PhotoImage(image)  
    bg_label = tk.Label(login_window, image=bg_image)  
    bg_label.image = bg_image  
    bg_label.place(x=0, y=0, relwidth=1, relheight=1)  
  
    login_window.title("Customer Login")  
    login_window.geometry("500x500")
```

```
tk.Label(login_window, text="Username",font=("Helvetica", 20)).pack(pady=50)
username_entry = tk.Entry(login_window)
username_entry.pack()
```

```
tk.Label(login_window, text="Password",font=("Helvetica", 20)).pack(pady=50)
password_entry = tk.Entry(login_window, show="*")
password_entry.pack()
```

```
def login():
    username = username_entry.get().strip()
    password = password_entry.get().strip()
```

```
    result = fetch_all("SELECT * FROM customers WHERE username=%s AND
password=%s", (username, password))
```

```
    if result:
```

```
        login_window.destroy()
        show_customer_dashboard(result[0])
```

```
    else:
```

```
        messagebox.showerror("Login Failed", "Invalid credentials")
```

```
tk.Button(login_window, text="Login", command=login).pack(pady=30)
```

```
def show_customer_dashboard(customer):
```

```
    window = tk.Toplevel()
```

```
    window.title(f"Customer Dashboard - {customer['name']}")
```

```
window.attributes('-fullscreen',True)
```

```
tk.Button(window, text="Home", command=lambda: [window.destroy(),  
os.system("python main.py")], bg="orange", fg="white", font=("Arial",  
12)).pack(pady=10)
```

```
style = ttk.Style()
```

```
style.theme_use("default")
```

```
# Change Treeview background
```

```
style.configure("Treeview",  
background="#e8f5e9",    # light green  
foreground="black",  
rowheight=25,  
fieldbackground="#e8f5e9")
```

```
# Change selected row highlight color
```

```
style.map("Treeview",  
background=[("selected", "#a5d6a7")])
```

```
notebook = ttk.Notebook(window)
```

```
notebook.pack(fill=tk.BOTH, expand=True)
```

```
browse_tab = ttk.Frame(notebook)
```

```
wishlist_tab = ttk.Frame(notebook)
```

```
notebook.add(browse_tab, text="Browse Products")
```

```
notebook.add(wishlist_tab, text="My Wishlist")
```

```

search_frame = tk.Frame(browse_tab)
search_frame.pack(pady=10)

tk.Label(search_frame, text="Search: ").grid(row=0, column=0, padx=5)
search_entry = tk.Entry(search_frame)
search_entry.grid(row=0, column=1, padx=5)

result_frame = tk.Frame(browse_tab)
result_frame.pack(fill=tk.BOTH, expand=True)

columns = ["ID", "Username", "Name", "Product", "Contact", "Tag", "Location"]
tree = ttk.Treeview(result_frame, columns=columns, show="headings")
for col in columns:
    tree.heading(col, text=col)
    tree.column(col, width=120)
tree.pack(fill=tk.BOTH, expand=True)

def display_results(results):
    for row in tree.get_children():
        tree.delete(row)
    for f in results:
        tree.insert("", "end", values=(
            f.get("id"), f.get("username"), f.get("name"), f.get("product"),
            f.get("contact"), f.get("tag"), f.get("location")
        ))

```

```

def search():
    q = search_entry.get().strip().lower()
    results = fetch_all("""
        SELECT * FROM farmers
        WHERE LOWER(tag) LIKE %s OR LOWER(location) LIKE %s OR
LOWER(product) LIKE %s
        """, (f"%{q}%", f"%{q}%", f"%{q}%"))
    display_results(results)

def view_selected():
    selected = tree.selection()
    if not selected:
        tk.Label(info_window, text= "Please select a farmer", font=("Arial",
12)).pack(pady=5)
        return

    row = tree.item(selected[0])['values']
    info_window = tk.Toplevel(window)
    info_window.title("Farmer Info")
    info_window.geometry("400x300")

    fields = tree["columns"]
    for i, field in enumerate(fields):
        tk.Label(info_window, text=f"{field.capitalize()}: {row[i]}", font=("Arial",
12)).pack(pady=5)

```



```

def save_to_wishlist():
    selected = tree.selection()
    if not selected:
        tk.Label(info_window, text="Please select a farmer", font=("Arial",
12)).pack(pady=5)
        return

    row = tree.item(selected[0])['values']
    info_window = tk.Toplevel(window)
    info_window.title("WishList Info")
    info_window.geometry("350x100")

    farmer_id = row[0]
    customer_id = customer['id']
    existing = fetch_all("SELECT * FROM wishlist WHERE customer_id=%s AND
farmer_id=%s", (customer_id, farmer_id))
    if existing:
        tk.Label(info_window, text=f"{row[2]}'s product is already in your wishlist.",
font=("Arial", 12)).pack(pady=5)
    else:
        execute_query("INSERT INTO wishlist (customer_id, farmer_id) VALUES
(%s, %s)", (customer_id, farmer_id))
        tk.Label(info_window, text= f"{row[2]}'s product is added to your wishlist.",
font=("Arial", 12)).pack(pady=5)
        load_wishlist()

    action_frame = tk.Frame(browse_tab)
    action_frame.pack(pady=5)

```

```

tk.Button(action_frame, text="View Selected", command=view_selected,
bg="blue", fg="white").pack(side=tk.LEFT, padx=5)

tk.Button(action_frame, text="Save to Wishlist", command=save_to_wishlist,
bg="green", fg="white").pack(side=tk.LEFT, padx=5)

tk.Button(search_frame, text="Search", command=search).grid(row=0, column=2,
padx=5)

display_results(fetch_all("SELECT * FROM farmers"))

wishlist_tree = ttk.Treeview(wishlist_tab, columns=columns, show="headings")
for col in columns:
    wishlist_tree.heading(col, text=col)
    wishlist_tree.column(col, width=120)
wishlist_tree.pack(fill=tk.BOTH, expand=True, padx=10, pady=10)

def load_wishlist():
    for row in wishlist_tree.get_children():
        wishlist_tree.delete(row)
    results = fetch_all("""
        SELECT f.* FROM farmers f
        JOIN wishlist w ON f.id = w.farmer_id
        WHERE w.customer_id = %s
        """, (customer['id'],))
    for f in results:
        wishlist_tree.insert("", "end", values=(
            f.get("id"), f.get("username"), f.get("name"), f.get("product"),
            f.get("contact"), f.get("tag"), f.get("location")

```

```
))
```

```
load_wishlist()
```

```
def remove_from_wishlist():
```

```
    selected = wishlist_tree.selection()
```

```
    if not selected:
```

```
        tk.Label(info_window, text= "Please select a farmer", font=("Arial",  
12)).pack(pady=5)
```

```
        return
```

```
    row = wishlist_tree.item(selected[0])['values']
```

```
    info_window = tk.Toplevel(window)
```

```
    info_window.title("Wishlist info")
```

```
    info_window.geometry("350x100")
```

```
    farmer_id = row[0]
```

```
    customer_id = customer['id']
```

```
    execute_query("DELETE FROM wishlist WHERE customer_id=%s AND  
farmer_id=%s", (customer_id, farmer_id))
```

```
    tk.Label(info_window, text= f' {row[2]} 's product removed from wishlist.",  
font=("Arial", 12)).pack(pady=5)
```

```
    load_wishlist()
```

```
    tk.Button(wishlist_tab, text="Remove from Wishlist",  
command=remove_from_wishlist, bg="red", fg="white").pack(pady=5)
```

4.farmer.py

```
import tkinter as tk

from tkinter import messagebox

from db import fetch_all, execute_query

import os

from PIL import Image,ImageTk


def open_farmer_interface():

    login_window = tk.Toplevel()

    image = Image.open(r"loginbg.jpg")

    image = image.resize((500,500), Image.Resampling.LANCZOS)

    bg_image = ImageTk.PhotoImage(image)

    bg_label = tk.Label(login_window, image=bg_image)

    bg_label.image = bg_image

    bg_label.place(x=0, y=0, relwidth=1, relheight=1)


    login_window.title("Farmer Login")

    login_window.geometry("500x500")


    tk.Label(login_window, text="Username",font=("Helvetica", 20)).pack(pady=50)

    username_entry = tk.Entry(login_window)

    username_entry.pack()


    tk.Label(login_window, text="Password",font=("Helvetica", 20)).pack(pady=50)

    password_entry = tk.Entry(login_window, show="*")
```

```
password_entry.pack()
```

```
def login():
```

```
    username = username_entry.get().strip()
```

```
    password = password_entry.get().strip()
```

```
    result = fetch_all("SELECT * FROM farmers WHERE username=%s AND  
password=%s", (username, password))
```

```
    if result:
```

```
        login_window.destroy()
```

```
        show_farmer_info(result[0])
```

```
    else:
```

```
        messagebox.showerror("Error", "Invalid credentials")
```

```
tk.Button(login_window, text="Login", command=login).pack(pady=30)
```

```
def show_farmer_info(farmer):
```

```
    window = tk.Toplevel()
```

```
    window.title("Farmer Dashboard")
```

```
    window.attributes('-fullscreen', True)
```

```
    image = Image.open(r"farmerbg.jpg")
```

```
    image = image.resize((1600,900), Image.Resampling.LANCZOS)
```

```
    bg_image = ImageTk.PhotoImage(image)
```

```
    bg_label = tk.Label(window, image=bg_image)
```

```
    bg_label.image = bg_image
```

```
bg_label.place(x=0, y=0, relwidth=1, relheight=1)
```

```
tk.Button(window, text="Home", command=lambda: [window.destroy(),  
os.system("python main.py")], bg="orange", fg="white", font=("Arial",  
12)).pack(pady=10)
```

```
fields = ["product", "contact", "tag", "location"]
```

```
entries = {}
```

```
for field in fields:
```

```
    tk.Label(window, text=f"{field.capitalize()}", font=("Helvetica",  
20)).pack(pady=20)
```

```
    entry = tk.Entry(window, width=30)
```

```
    entry.insert(0, farmer.get(field, ""))
```

```
    entry.pack()
```

```
    entries[field] = entry
```

```
def update_info():
```

```
    info_window = tk.Toplevel(window)
```

```
    info_window.title("Wishlist info")
```

```
    info_window.geometry("350x100")
```

```
    updates = {f: entries[f].get().strip() for f in fields}
```

```
    query = "UPDATE farmers SET product=%s, contact=%s, tag=%s, location=%s  
WHERE id=%s"
```

```
    execute_query(query, (updates["product"], updates["contact"], updates["tag"],  
updates["location"], farmer["id"]))
```

```
tk.Label(info_window, text= "Information updated.", font=("Arial",  
12)).pack(pady=5)
```

```
tk.Button(window, text="Update Info", command=update_info,font=("Helvetica",  
20),bg="#4CAF50", fg="white").pack(pady=30)
```

5. register.py

```
import tkinter as tk
```

```
from tkinter import messagebox
```

```
from db import execute_query,fetch_all
```

```
def open_registration_window():
```

```
    window = tk.Toplevel()
```

```
    window.title("Register")
```

```
    window.geometry("600x700")
```

```
tk.Label(window, text="Register As", font=("Arial", 20, "bold")).pack(pady=30)
```

```
role_var = tk.StringVar(value="farmer")
```

```
tk.Radiobutton(window, text="Farmer", variable=role_var,  
value="farmer").pack(pady=10)
```

```
tk.Radiobutton(window, text="Customer", variable=role_var,  
value="customer").pack(pady=10)
```

```
entries = {}
```

```
fields = ["Username", "Password", "Name"]
```

```
for field in fields:
```

```

tk.Label(window, text=field).pack()

entry = tk.Entry(window)

entry.pack()

entries[field.lower()] = entry


additional = {
    "farmer": ["Product (with price)", "Contact", "Tag", "Location"],
    "customer": []
}

additional_entries = {}

dynamic_frame = tk.Frame(window)

dynamic_frame.pack(pady=30)


def update_form():
    for widget in dynamic_frame.winfo_children():
        widget.destroy()

    additional_entries.clear()

    for field in additional[role_var.get()]:
        tk.Label(dynamic_frame, text=field).pack()

        entry = tk.Entry(dynamic_frame)

        entry.pack()

        additional_entries[field.lower()] = entry


role_var.trace("w", lambda *args: update_form())

update_form()

```



```
def register():
```

```
    username = entries["username"].get().strip()
```

```
    password = entries["password"].get().strip()
```

```
    name = entries["name"].get().strip()
```

```
    role = role_var.get()
```

```
    if not (username and password and name):
```

```
        messagebox.showerror("Error", "Please fill all basic fields")
```

```
        return
```

```
    if role == "farmer":
```

```
        product = additional_entries["product (with price)"].get().strip()
```

```
        contact = additional_entries["contact"].get().strip()
```

```
        tag = additional_entries["tag"].get().strip()
```

```
        location = additional_entries["location"].get().strip()
```

```
    if not all([product, contact, tag, location]):
```

```
        messagebox.showerror("Error", "Please fill all farmer fields")
```

```
    if not username or not password or not name:
```

```
        messagebox.showwarning("Input Error", "All fields are required.")
```

```
        return
```

```
    existing = fetch_all("SELECT * FROM farmers WHERE username=%s",  
(username,))
```

```
    if existing:
```

```
messagebox.showerror("Registration Failed", "Username already exists!")  
return
```

```
query = "INSERT INTO farmers (username, password, name, product, contact,  
tag, location) VALUES (%s, %s, %s, %s, %s, %s, %s)"
```

```
values = (username, password, name, product, contact, tag, location)
```

```
else:
```

```
existing = fetch_all("SELECT * FROM customers WHERE username=%s",  
(username,))
```

```
if existing:
```

```
messagebox.showerror("Registration Failed", "Username already exists!")  
return
```

```
query = "INSERT INTO customers (username, password, name) VALUES  
(%s, %s, %s)"
```

```
values = (username, password, name)
```

```
try:
```

```
execute_query(query, values)
```

```
messagebox.showinfo("Success", f"{role.capitalize()} registered successfully")
```

```
window.destroy()
```

```
except Exception as e:
```

```
messagebox.showerror("Database Error", str(e))
```

```
tk.Button(window, text="Register", bg="#4CAF50", fg="white",  
command=register).pack(pady=30)
```

6. main.py

```
import tkinter as tk

from admin import open_admin_dashboard
from customer import open_customer_interface
from farmer import open_farmer_interface
from register import open_registration_window
from PIL import Image,ImageTk

import os,sys


def main():
    root = tk.Tk()

    image = Image.open(r"bg.jpeg")
    image = image.resize((1550,1020), Image.Resampling.LANCZOS)
    bg_image = ImageTk.PhotoImage(image)
    bg_label = tk.Label(root, image=bg_image)
    bg_label.image = bg_image
    bg_label.place(x=0, y=0, relwidth=1, relheight=1)
    root.title("Farm-It | Login")
    root.attributes('-fullscreen',True)
    root.configure(bg="#f0fff0")


    tk.Label(root, text="Welcome to Farm-It", font=("Helvetica", 50, "bold"),
bg="#228B22",fg="#FFFFFF").pack(pady=90)


    tk.Button(root, text="Admin Login", width=25, font=("Helvetica", 20),
bg="#4CAF50", fg="white", command=open_admin_dashboard).pack(pady=30)
```

```
tk.Button(root, text="Customer Login", width=25, font=("Helvetica", 20),  
bg="#2196F3", fg="white", command=open_customer_interface).pack(pady=30)
```

```
tk.Button(root, text="Farmer Login", width=25, font=("Helvetica", 20),  
bg="#FF9800", fg="white", command=open_farmer_interface).pack(pady=30)
```

```
tk.Button(root, text="Register", width=25, font=("Helvetica", 20), bg="#9C27B0",  
fg="white", command=open_registration_window).pack(pady=30)
```

```
tk.Button(root, text="Exit", width=25, font=("Helvetica", 20),bg="#f44336",  
fg="white", command=root.destroy).pack(pady=5)
```

```
root.mainloop()
```

```
main()
```

7. SQL Commands

```
CREATE DATABASE IF NOT EXISTS farmit;
```

```
USE farmit;
```

```
CREATE TABLE admins (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    username VARCHAR(50),  
    password VARCHAR(50)  
);
```

```
CREATE TABLE IF NOT EXISTS farmers (
```

```
id INT AUTO_INCREMENT PRIMARY KEY,  
username VARCHAR(50) NOT NULL,  
password VARCHAR(50) NOT NULL,  
name VARCHAR(100) NOT NULL,  
product VARCHAR(100),      -- nullable  
contact VARCHAR(20),  
tag VARCHAR(50),  
location VARCHAR(100)  
);
```

```
CREATE TABLE customers (  
id INT AUTO_INCREMENT PRIMARY KEY,  
username VARCHAR(50),  
password VARCHAR(50),  
name VARCHAR(100)  
);
```

```
INSERT INTO admins (username, password) VALUES ('admin', 'admin123');
```

```
INSERT INTO farmers (username, password, name, product, contact, tag, location)  
VALUES  
(f1', 'pass1', 'Raj Patel', 'Wheat 100/kg', '9876543210', '#grains', 'Ahmedabad'),  
(f2', 'pass2', 'Lakshmi Reddy', 'Rice 90/kg', '9123456789', '#cereals', 'Hyderabad'),  
(f3', 'pass3', 'John Singh', 'Mango 60/kg', '9988776655', '#fruits', 'Nagpur'),  
(f4', 'pass4', 'Meena Sharma', 'Tomato 30/kg', '9871234567', '#vegetables', 'Jaipur'),
```

```
('f5', 'pass5', 'Kiran Das', NULL, NULL, NULL, NULL); -- Farmer registered but  
hasn't added details yet
```

```
INSERT INTO customers (username, password, name)  
VALUES  
( 'c1', 'cust1', 'Amit Shah'),  
( 'c2', 'cust2', 'Sneha Iyer'),  
( 'c3', 'cust3', 'Rahul Dev'),  
( 'c4', 'cust4', 'Priya Mehta'),  
( 'c5', 'cust5', 'Naveen Kumar');
```

```
CREATE TABLE IF NOT EXISTS wishlist (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    customer_id INT,  
    farmer_id INT,  
    FOREIGN KEY (customer_id) REFERENCES customers(id),  
    FOREIGN KEY (farmer_id) REFERENCES farmers(id)  
);
```

```
ALTER TABLE customers ADD UNIQUE (username);  
ALTER TABLE farmers ADD UNIQUE (username);
```

8. CMD prompt for .exe creation

```
cd C:\farm_it
```

```
pip install pyinstaller
```

```
pyinstaller --onefile --windowed ^
```

```
--add-data "bg.jpeg;." ^
```

```
--add-data "farmerbg.jpg;." ^
```

```
--add-data "loginbg.jpg;." ^
```

```
--add-data "admin.py;." ^
```

```
--add-data "customer.py;." ^
```

```
--add-data "farmer.py;." ^
```

```
--add-data "register.py;." ^
```

```
--add-data "db.py;." ^
```

```
main.py
```

Application Flow:

1. User Selection

- The application begins with a home screen offering three login options: **Farmer Login, Customer Login, and Admin Login.**

2. Farmer Login

- Farmers log in using their credentials.
- After logging in, they can add, edit, or update their product details (name, price, tag, contact, and location).
- The updated data is stored in the MySQL database.

3. Customer Login

- Customers log in to browse products.
- They can search and filter products based on **tags, price range, product name, and location.**
- Customers can add products to their wishlist for easy access later.

4. Admin Login

- Admins can view all registered farmers and customers in a tabular format.
- Admins have the authority to delete or modify any farmer profile or product listing to maintain system integrity.

5. Database Interaction

- Every action (login, product search, wishlist management, etc.) communicates with the MySQL database to fetch or store information securely.

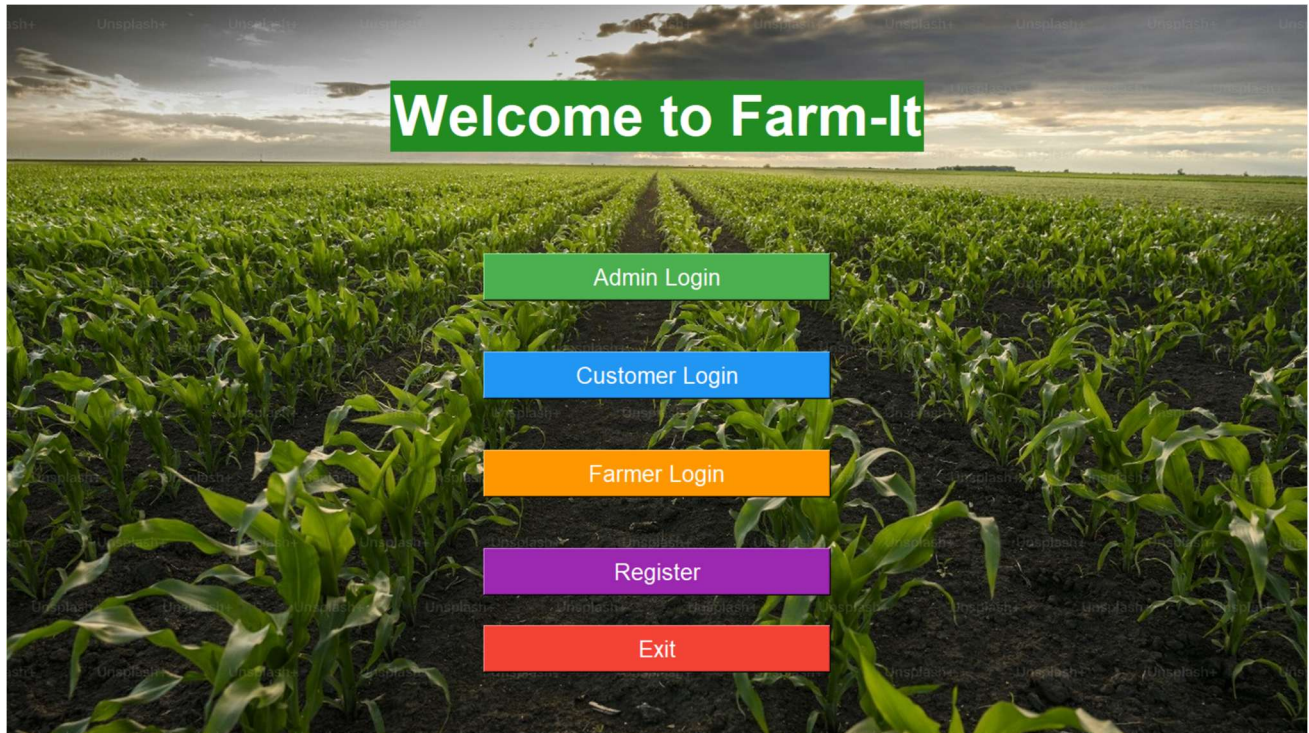
6. Exit and Restart

- The user can return to the home screen or exit the application using dedicated buttons.

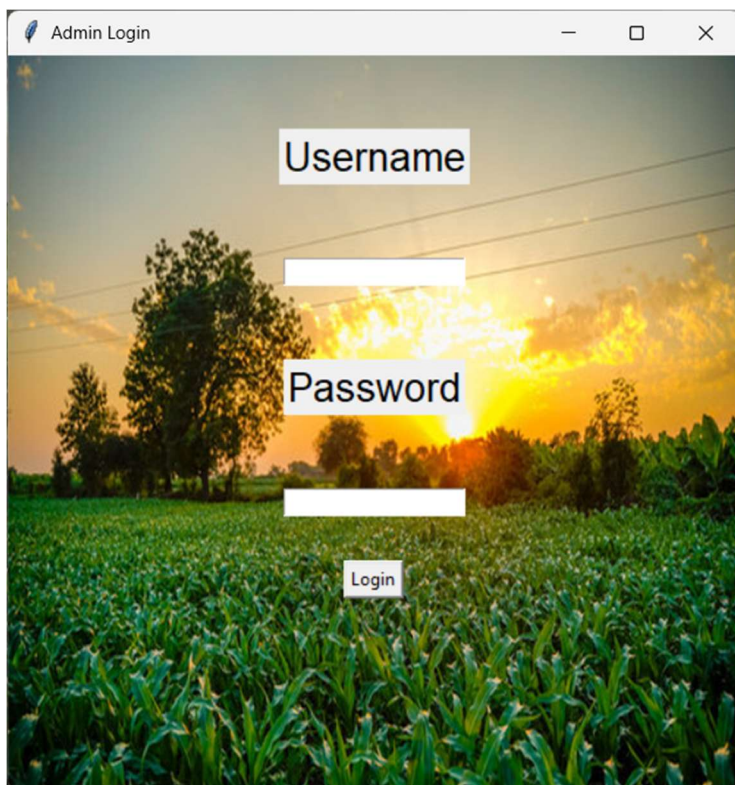
This flow ensures smooth navigation for all types of users while maintaining data integrity and providing a seamless user experience.

Home Screens:

1. main window



2. login window



3. Admin dashboard

Home									
Farmers	Customers								
	id	username	password	name	product	contact	tag	location	
1	f1		pass1	Raj Patel	Wheat 100/kg	9876543210	#grains	Ahmedabad	
2	f2		pass2	Lakshmi Reddy	Rice 90/kg	9123456789	#cereals	Hyderabad	
3	f3		pass3	John Singh	Mango 60/kg	9988776655	#fruits	Nagpur	
4	f4		pass4	Meena Sharma	Tomato 30/kg	9871234567	#vegetables	Jaipur	
6	f6		pass6	arjun	oats	1234567891	#food	chennai	

Edit Selected

Delete Selected

4. Customer dashboard

Home

Browse ProductsMy Wishlist

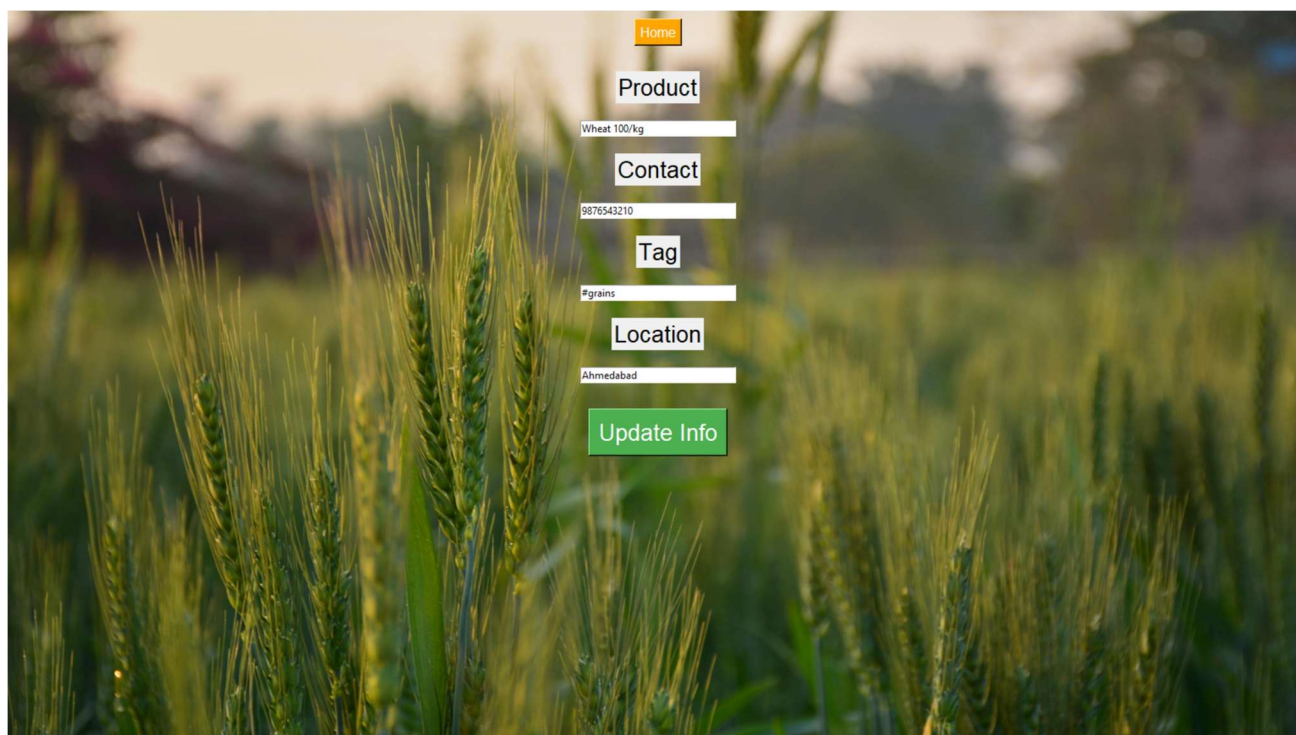
Search: Search

ID	Username	Name	Product	Contact	Tag	Location
1	f1	Raj Patel	Wheat 100/kg	9876543210	#grains	Ahmedabad
2	f2	Lakshmi Reddy	Rice 90/kg	9123456789	#cereals	Hyderabad
3	f3	John Singh	Mango 60/kg	9988776655	#fruits	Nagpur
4	f4	Meena Sharma	Tomato 30/kg	9871234567	#vegetables	Jaipur

View Selected

Save to Wishlist

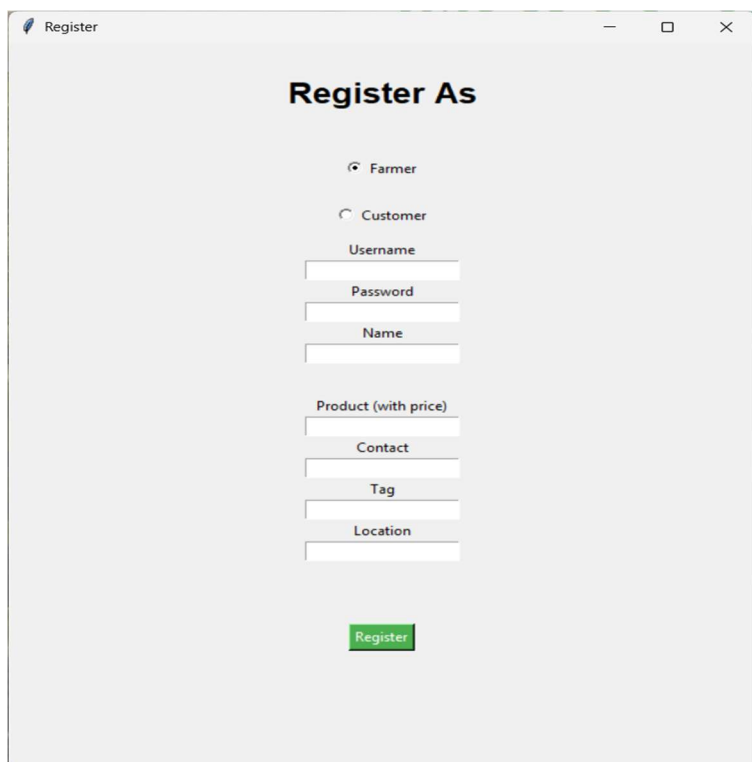
5. Farmer dashboard



The image shows a farmer dashboard form overlaid on a background of a wheat field. The form includes the following elements:

- Home** (orange button)
- Product** (label)
- Wheat 100/kg (text input)
- Contact** (label)
- 9876543210 (text input)
- Tag** (label)
- #grains (text input)
- Location** (label)
- Ahmedabad (text input)
- Update Info** (green button)

6. Register window



The image shows a screenshot of a 'Register' window. The window title is 'Register'. The main heading is 'Register As'. There are two radio buttons for 'Farmer' (selected) and 'Customer'. Below the radio buttons are several text input fields: 'Username', 'Password', 'Name', 'Product (with price)', 'Contact', 'Tag', and 'Location'. At the bottom is a green 'Register' button.

Register

Register As

☒ Farmer
☐ Customer

Username
Password
Name
Product (with price)
Contact
Tag
Location

Register

Register

— □ ×

Register As

☐ Farmer

☒ Customer

Username

Password

Name

Register

Report Module :

1.admin screen

Editing farmer detail

Home

Farmers	id	username	password	name	product	contact	tag	location
1	f1	pass1	Raj Patel	Wheat 100/kg	9876543210	#grains	Ahmedabad	
2	f2	pass2	Lakshmi Reddy	Rice 90/kg	9123456789	#cereals	Hyderabad	
3	f3	pass3	John Singh	Mango 60/kg	9988776655	#fruits	Nagpur	
4	f4	pass4	Meena Sharma	Tomato 30/kg	9871234567	#vegetables	Jaipur	
5	f5	pass5	Kiran Das	None	None	None	None	

Edit Selected

Delete Selected

Deleting farmer(here kiran)

Home

FarmersCustomers

id	username	password	name	product	contact	tag	location
1	f1	pass1	Raj Patel	Wheat 100/kg	9876543210	#grains	Ahmedabad
2	f2	pass2	Lakshmi Reddy	Rice 90/kg	9123456789	#cereals	Hyderabad
3	f3	pass3	John Singh	Mango 60/kg	9988776655	#fruits	Nagpur
4	f4	pass4	Meena Sharma	Tomato 30/kg	9871234567	#vegetables	Jaipur

Edit Selected

Delete Selected

Deleting customer(here c5)

Home

Farmers	Customers			
	id	username	password	name
1	c1	cust1		Amit Shah
2	c2	cust2		Sneha Iyer
3	c3	cust3		Rahul Dev
4	c4	cust4		Priya Mehta

Delete Selected

Home

Farmers	Customers			
	id	username	password	name
1	c1	cust1		Amit Shah
2	c2	cust2		Sneha Iyer
3	c3	cust3		Rahul Dev
4	c4	cust4		Priya Mehta

Delete Selected

2.customer screen

Viewing farmer detail

Farmer Info

Id: 4

Username: f4

Name: Meena Sharma

Product: Tomato 30/kg

Contact: 9871234567

Tag: #vegetables

Location: Jaipur

	Name	Product	Contact	Tag	Location
1	Patel	Wheat 100/kg	9876543210	#grains	Ahmedabad
2	Sharma	Rice 90/kg	9123456789	#cereals	Hyderabad
3	Singh	Mango 60/kg	9988776655	#fruits	Nagpur
4	Meena Sharma	Tomato 30/kg	9871234567	#vegetables	Jaipur

View Selected Save to Wishlist

search

Search: tom

ID	Username	Name	Product	Contact	Tag	Location
4	f4	Meena Sharma	Tomato 30/kg	9871234567	#vegetables	Jaipur

View Selected Save to Wishlist

Saving farmer to wishlist

Home

Browse Products

My Wishlist

Search:

Search

	ID	Username	Name	Product	Contact	Tag	Location
1				Wheat 100/kg	9876543210	#grains	Ahmedabad
2			Reddy	Rice 90/kg	9123456789	#cereals	Hyderabad
3			gh	Mango 60/kg	9988776655	#fruits	Nagpur
4			Sharma	Tomato 30/kg	9871234567	#vegetables	Jaipur

WishList Info

Meena Sharma's product is added to your wishlist

View Selected

Save to Wishlist

Home

Browse Products

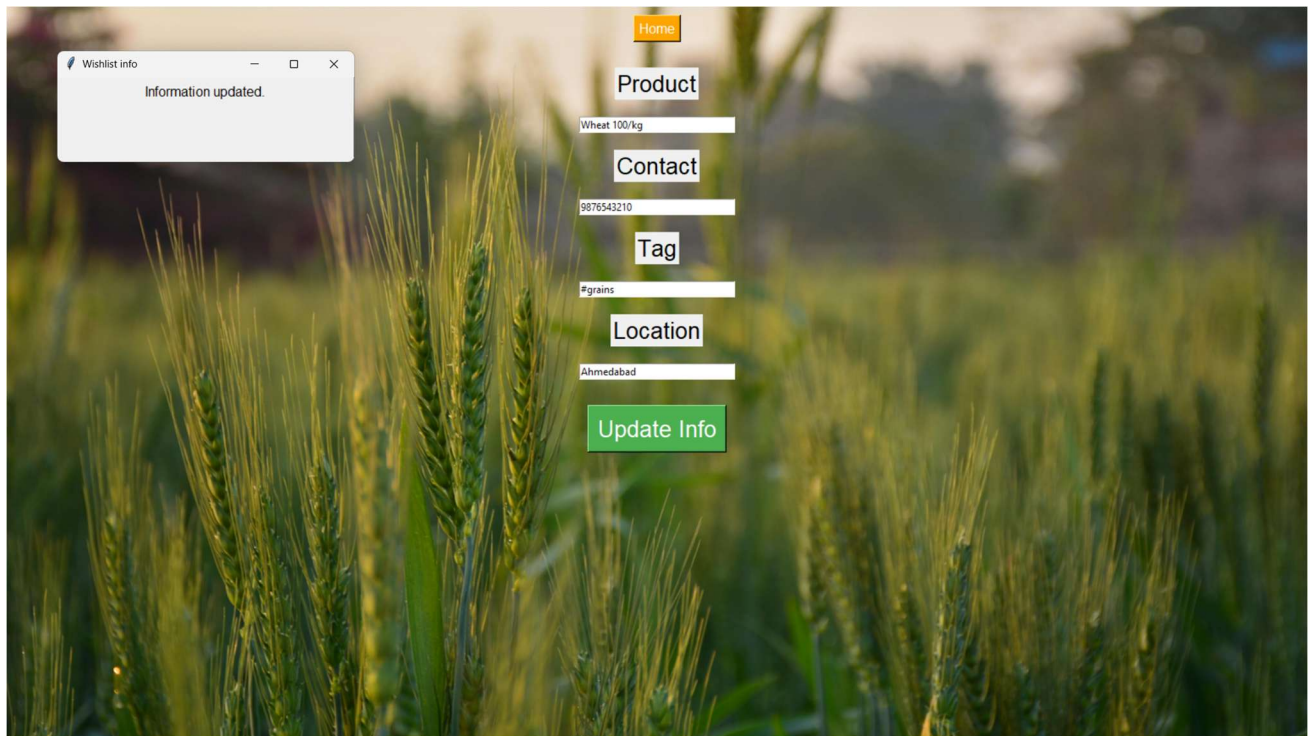
My Wishlist

	ID	Username	Name	Product	Contact	Tag	Location
4	14		Meena Sharma	Tomato 30/kg	9871234567	#vegetables	Jaipur

Remove from Wishlist

3.farmer screen

Updating farmer information



4.Register screen

Registering as farmer

Register As

☒ Farmer
☐ Customer

Username
f6

Password
pass6

Name
arjun

Product (with price)
oats

Contact
1234567891

Tag
#food

Location
chennai

Register

Admin Login

Customer Login

Farmer Login

Register

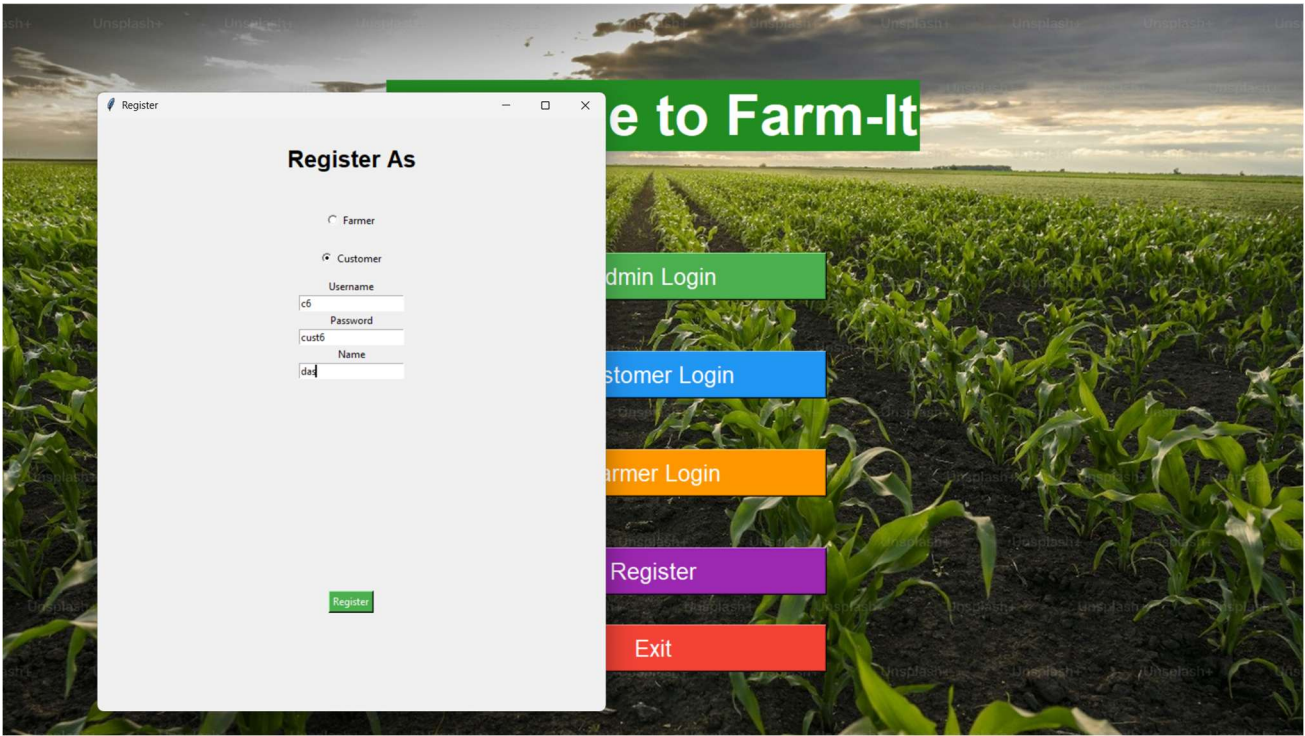
Exit

Home									
Farmers									
Customers									
	id	username	password	name	product	contact	tag	location	
1	f1	pass1	Raj Patel	Wheat 100/kg	9876543210	#grains	Ahmedabad		
2	f2	pass2	Lakshmi Reddy	Rice 90/kg	9123456789	#cereals	Hyderabad		
3	f3	pass3	John Singh	Mango 60/kg	9988776655	#fruits	Nagpur		
4	f4	pass4	Meena Sharma	Tomato 30/kg	9871234567	#vegetables	Jaipur		
6	f6	pass6	arjun	oats	1234567891	#food	chennai		

Edit Selected

Delete Selected

Registrering as customer



Home				
Farmers	Customers			
	id	username	password	name
1	c1	cust1		Amit Shah
2	c2	cust2		Sneha Iyer
3	c3	cust3		Rahul Dev
4	c4	cust4		Priya Mehta
6	c6	cust6		das

Delete Selected

File part of this project :

Python files:



db.py



admin.py



customer.py



farmer.py



register.py



main.py

Exe File:



farm_it.exe

Bibliography:

1. <https://www.chatgpt.com/> – reference for base code
2. <https://www.pinterest.com/> – bg image
3. <https://docs.python.org/3/library/tkinter.html> - reference for use of tkinter module